

PA – 18241

REST-API

Programação Aplicada

Ricardo Marau
marau@ua.pt



ESTGA

universidade de aveiro

Motivação

- ◉ REST API
 - ◉ Representational State Transfer
 - ◉ Application Programming Interface
- ◉ Desenvolvimento Web
- ◉ Aplicação em IoT
- ◉ Interface de bases de dados

Motivação

Desenvolvimento Web

- Comunicação Cliente-Servidor
 - Normalização (standard) da comunicação entre diferentes sistemas.
 - Frameworks para clientes comunicam com servidores para troca de informação.
- Independência de linguagem e plataformas
 - Comunicação baseada em http

Motivação

Desenvolvimento Web

- Escalabilidade e Modularidade
 - Concept of resources URI (Uniform Resource Identifier)
 - Possibilidade de lidar com grande número de recursos de forma escalar.
 - Permite modificar acesso a recursos sem afetar outros recursos.
- Comunicação Stateless
 - Cada pedido ao servidor é auto contido em termos de contexto.
 - O servidor não tem de manter o estado relativo ao pedido anterior.

Motivação

Desenvolvimento Web

- ◉ Uniformização
 - ◉ A interface utiliza comandos HTTP (GET POST PUT DELETE) que mapeiam naturalmente com as operações CRUD mais comuns:
 - ◉ Create, Read, Update, Delete
 - ◉ Json - Normalmente a troca de dados em HTTP é feita através de texto json
 - ◉ Melhora interoperabilidade e facilidade de utilização.
- ◉ Caching...

Motivação IoT

- Comunicação entre Dispositivos
 - API REST fornece ligação padronizada entre muitos dispositivos
 - Cada dispositivo comunica com o servidor via HTTP nos endpoints/recursos correspondentes.
- Acesso e gestão dos dados
 - A API REST fornece fácil acesso aos vários endpoints de dados: ler, escrever, atualizar, backup, etc..
- Interoperabilidade
 - O servidor REST pode ligar-se de forma simples a outros componentes, servindo de gateway para outros sistemas como a cloud.

Motivação IoT

- ◉ Escalabilidade
 - ◉ Gestão de dispositivos IoT e recursos/endpoints é escalável e flexível
 - ◉ A utilização de http permite mesmo a escalabilidade física de processamento de grandes volumes de dados e processamento.
 - ◉ Flexível – porque pode ser adaptada dinamicamente.
- ◉ Segurança
 - ◉ As APIs REST podem implementar mecanismos de autenticação e autorização
 - ◉ OAuth

Motivação

Interface de bases de dados

- Padronização e universalidade
- API de acesso comum
 - Camada de abstração para manipulação e acesso aos dados
- Abstração da implementação
 - A tecnologia de apoio à DB torna-se irrelevante do ponto de vista do acesso.
- Flexibilidade e Escalabilidade...
- Segurança e controlo...
- Integração com outras aplicações...

Componentes REST

- Recursos
 - Informação ou entidade que pode ser manipulada
 - Objeto físico, documento digital, entrada numa db, etc
 - Exemplo: utilizadores, produtos, pedidos, ...
- URIs (Uniform Resource Identifiers)
 - Permitem identificar de forma única um recurso
 - Similares a URLs (Uniform Resource Locators), mas mais genéricos.
 - Exemplo: `"/users/123"` pode ser um URI que representa um user específico com ID 123

Componentes REST

- Métodos HTTP (Verbos / CRUD)
 - GET: Obter uma representação do recurso.
 - POST: Criar novo recurso. É utilizado para enviar dados a serem processados/armazenados, resultando num novo recurso.
 - PUT: Atualizar um recurso existente.
 - PATCH: Atualizar um recurso existente, parcialmente.
 - DELETE: Remove um recurso.
- Representação:
 - A representação dos recursos pode ser variada: Json, XML, HTML, etc
 - Nos cabeçalhos do HTTP request, pode-se identificar o formato pretendido do servidor.

Python-Flask

- ◉ Microframework
 - ◉ Suporte básico de acesso
 - ◉ Abordagem minimalista que pode ser adjuvada com extensões.
- ◉ Roteamento (routing) simples
 - ◉ Mapear URIs para funções python
- ◉ Integrações e extensões:
 - ◉ Flask-RESTful
 - ◉ Flask-SQLAlchemy
 - ◉ Flask-Login
 - ◉ ...

```
#!/usr/bin/env python
# encoding: utf-8
import json
from flask import Flask
app = Flask(__name__)
@app.route('/')
def index():
    return json.dumps({'name': 'alice',
                       'email': 'alice@outlook.com'})

app.run()
```

localhost:5000

⏮ ⏭ ⏲ 📖 ! localhost:5000

`{"name": "alice", "email": "alice@outlook.com"}`

▼ General

Request URL: `http://localhost:5000/`
Request Method: GET
Status Code: 🟢 200 OK
Remote Address: 127.0.0.1:5000

▼ Response Headers [view source](#)

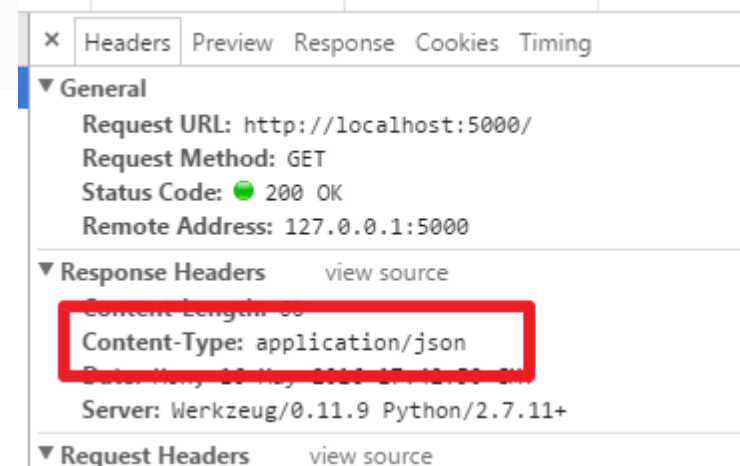
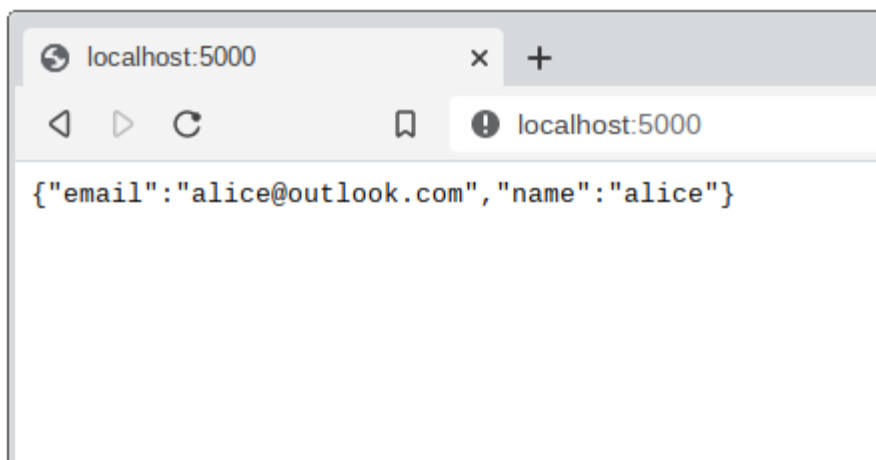
`Content-Type: text/html; charset=utf-8`

Date: Mon, 10 May 2016 17:59:11 GMT
Server: Werkzeug/0.11.9 Python/2.7.11+

<https://pythonbasics.org/flask-rest-api/>

```
#!/usr/bin/env python
# encoding: utf-8
import json
from flask import Flask, jsonify
app = Flask(__name__)
@app.route('/')
def index():
    return jsonify({'name': 'alice',
                    'email': 'alice@outlook.com'})

app.run()
```



<https://pythonbasics.org/flask-rest-api/>

Desafio

- ◉ Implemente um servidor REST para fazer a ligação com a base de dados do trabalho anterior.
- ◉ Deverá ser possível adicionar
 - ◉ Consultar um aluno
 - ◉ Adicionar um aluno
- ◉ Flask + SQLAlchemy
 - ◉ Exemplo: <https://pythonbasics.org/flask-sqlalchemy/>